

Universidade Federal de Santa Catarina
Curso de Ciência da Computação

Automação e Auditoria de Segurança Contínua: Implementação de Pipeline DevSecOps em Sistema de Helpdesk

Eduardo Achar - 23102448

22 de junho de 2026

Resumo

Este relatório descreve o desenvolvimento de uma aplicação de Helpdesk contêinerizada e a implementação de uma esteira automatizada de DevSecOps. O projeto explora a integração de ferramentas de segurança nas fases de Integração Contínua e Entrega Contínua (CI/CD), contemplando análises de código estático (SAST), detecção de segredos, análise de dependências (SCA), verificação de Infraestrutura como Código (IaC) e testes dinâmicos (DAST). Os resultados demonstram a identificação e mitigação arquitetural de vulnerabilidades críticas, consolidando as boas práticas de desenvolvimento seguro.

Conteúdo

1	Descrição do Sistema e Ferramental	3
1.1	Plataforma de CI/CD e Integração DevSecOps	3
2	Evidências de Execução	3
2.1	Verificações Estáticas (Secrets, SCA e SAST)	4
2.2	Verificações de Infraestrutura (tfsec) e Ataque Dinâmico (ZAP)	4
3	Análise de Falsos Positivos e Alertas Irrelevantes	4
3.1	Information Disclosure - Suspicious Comments (ZAP)	4
3.2	Diretivas CSP unsafe-inline (ZAP)	5
3.3	Storable and Cacheable Content (ZAP)	5
3.4	Ponto Cego de Contexto: Credenciais no Banco de Dados (Semgrep e Gitleaks)	5
4	Identificação e Correção de Falhas Reais	5
4.1	Falha 1: Exposição de Rede e Falta de Criptografia At-Rest (tfsec)	5
4.2	Falha 2: Ausência de Cabeçalhos de Segurança (OWASP ZAP)	6
4.3	Falha 3: Superfície de Ataque Excessiva no Contêiner Web (Docker)	6
5	Decisões Arquiteturais e Desafios de CI/CD	7
6	Conclusão	7

1 Descrição do Sistema e Ferramental

O sistema auditado neste trabalho consiste em uma plataforma de Helpdesk (gerenciamento de chamados técnicos) concebida através de uma arquitetura de microsserviços. O ecossistema é formado por um Front-end construído em *React/Vite* (servido de forma otimizada via servidor *Nginx*), um Back-end implementado como uma API REST em *Node.js*, e persistência de dados em um banco relacional *PostgreSQL*. Para garantir o isolamento e a reprodutibilidade, toda a aplicação foi containerizada com *Docker* e provisionada na nuvem da *Amazon Web Services* (AWS) via *Terraform*.

Uma premissa fundamental e intencional deste projeto foi a separação de responsabilidades na engenharia do software. O código-fonte de negócio da aplicação foi gerado através de *Vibecoding* (desenvolvimento assistido por Inteligência Artificial). A geração de código primária por IA foca na entrega rápida de funcionalidades, mas frequentemente omite práticas robustas de segurança (como cabeçalhos HTTP estritos). Essa deficiência induzida serviu como o cenário ideal para comprovar a eficácia analítica da esteira de DevSecOps, que foi totalmente arquitetada e configurada de forma manual.

1.1 Plataforma de CI/CD e Integração DevSecOps

A automação foi orquestrada utilizando o **GitHub Actions**. O *pipeline* foi configurado para atuar como um *Quality Gate* inflexível a cada *push* na *branch* principal, integrando as cinco camadas obrigatórias de varredura estipuladas para este estudo:

- **Secret Detection:** *Gitleaks* (Auditoria do histórico em busca de credenciais vazadas).
- **SCA (Software Composition Analysis):** *npm audit* (Verificação de CVEs e falhas conhecidas na árvore de dependências de terceiros do Front-end e Back-end).
- **SAST (Static Application Security Testing):** *Semgrep* (Análise estática da sintaxe do código JS/React contra padrões inseguros).
- **IaC Scanning (Infraestrutura como Código):** *tfsec* (Validação estática de conformidade e *hardening* dos manifestos do Terraform).
- **DAST (Dynamic Application Security Testing):** *OWASP ZAP* (Ataque dinâmico automatizado aos contêineres levantados em tempo de execução na esteira).

O código-fonte, o histórico de *commits*, as configurações de infraestrutura e as *Issues* geradas pelos robôs de segurança são públicos e estão disponíveis para auditoria no seguinte repositório:

<https://github.com/dd-znh/helpdesk-system>.

2 Evidências de Execução

Para ratificar a execução do *pipeline*, destacam-se recortes das saídas textuais (*logs*) dos *Jobs* processados pela máquina virtual do GitHub Actions. A execução completa gerou relatórios dinâmicos que foram atrelados diretamente ao repositório via *Issues*.

2.1 Verificações Estáticas (Secrets, SCA e SAST)

O *job* primário encarregou-se da varredura do histórico Git, dependências Node.js e código-fonte, confirmando a ausência de chaves em texto claro e o saneamento das bibliotecas utilizadas.

Listing 1: Logs de aprovação das camadas de Código e Dependências

```
Run gitleaks/gitleaks-action@v2
  8 commits scanned.
  No leaks found.

Run semgrep/action@v1
  Scan completed successfully. No high-risk patterns found in JavaScript/Jsx logic.

Run npm audit --audit-level=moderate || true
  (Backend) found 0 vulnerabilities
  (Frontend) found 0 vulnerabilities
```

2.2 Verificações de Infraestrutura (tfsec) e Ataque Dinâmico (ZAP)

O *job* subsequente auditou o provisionamento simulado da AWS e engatilhou o ataque DAST. O *tfsec* e o *ZAP* encontraram com sucesso dezenas de vulnerabilidades, provando o valor da ferramenta, cujas correções serão detalhadas nas seções seguintes.

Listing 2: Captura da execução do IaC Scan e DAST

```
Run aquasecurity/tfsec-action@v1.0.0
  Result #1 CRITICAL Security group rule allows ingress from public internet.
  Result #5 HIGH Root block device is not encrypted.
  4 passed, 6 potential problem(s) detected.

Run zapproxy/action-baseline@v0.12.0
  WARN-NEW: Content Security Policy (CSP) Header Not Set [10038] x 3
  WARN-NEW: Missing Anti-clickjacking Header [10020] x 2
  Scanning process completed, starting to analyze the results!
  Process completed successfully and a new issue #1 has been created for the ZAP Scan.
```

3 Análise de Falsos Positivos e Alertas Irrelevantes

Sistemas automatizados de segurança (notoriamente ferramentas DAST) geram invariavelmente ruídos (*false positives*). O papel da engenharia de DevSecOps é promover a "aceitação de risco" baseada no contexto operacional da aplicação.

3.1 Information Disclosure - Suspicious Comments (ZAP)

A ferramenta relatou um suposto vazamento de informações decorrente de "comentários suspeitos" em arquivos estáticos (ex: `/assets/index-28w9bwLP.js`). Uma inspeção técnica revela que este é um falso positivo clássico. O ZAP escaneou os pacotes minificados pelo *Vite* (empacotador do React) e interpretou declarações de licenças de *software livre* e cabeçalhos estruturais inofensivos como dados sensíveis. O alerta foi ignorado com segurança.

3.2 Diretivas CSP `unsafe-inline` (ZAP)

Mesmo após as correções da Seção 4, o ZAP continuou a alertar sobre a presença de diretivas `unsafe-inline` na *Content Security Policy* (CSP). Embora bloqueios estritos de injeção direta sejam o ideal teórico, no contexto de *Single Page Applications* (SPA) modernas, o ecossistema injeta folhas de estilo CSS e *scripts* modulares diretamente na árvore do DOM em tempo real. Remover o permissivo `unsafe-inline` causaria a paralisação completa do Front-end. A mitigação exigiria infraestrutura avançada baseada em *nonces* aleatórios injetados no lado do servidor a cada requisição, um esforço desproporcional ao objetivo desta prova de conceito, motivando a aceitação técnica deste risco.

3.3 Storable and Cacheable Content (ZAP)

Alerta classificado como risco irrelevante (Informacional). O ZAP notificou que arquivos como CSS e SVG estão sendo retidos em *cache*. Em aplicações Web, o armazenamento em cache de componentes estáticos não-sensíveis é o comportamento exato e desejado do servidor Nginx para reduzir a latência e consumo de banda de rede do usuário final.

3.4 Ponto Cego de Contexto: Credenciais no Banco de Dados (Semgrep e Gitleaks)

Durante uma auditoria manual do código, observou-se que o *script* de inicialização do banco de dados (`database/init.sql`) continha um *hash bcrypt* predefinido para um usuário administrador, com a respectiva senha em texto claro ("admin123") descrita em um comentário. Notavelmente, nenhuma das ferramentas da esteira sinalizou este trecho como vulnerabilidade.

Esta ocorrência não é um erro da ferramenta, mas uma limitação arquitetural conhecida como "Cegueira de Contexto" (*Context Blindness*). O *Gitleaks* não classifica *hashes* unidirecionais como segredos absolutos e não correlaciona linguagem natural em comentários. Mais criticamente, a ineficácia na detecção evidencia uma limitação de escopo da própria configuração de SAST: como o *Semgrep* foi instanciado no *pipeline* utilizando o pacote de regras `p/javascript`, o motor de análise focou estritamente na árvore sintática do Front-end e Back-end, ignorando sistematicamente arquivos de extensão `.sql`. Este cenário comprova que a automação segmentada cria pontos cegos, exigindo configurações multilíngues robustas e não substituindo a revisão humana (*Code Review*).

4 Identificação e Correção de Falhas Reais

Esta seção demonstra o valor da cultura *Shift-Left Security*. As ferramentas revelaram falhas graves de arquitetura que foram interceptadas e refatoradas no código muito antes de chegarem à fase de implantação (*Deploy*).

4.1 Falha 1: Exposição de Rede e Falta de Criptografia At-Rest (tfsec)

A Vulnerabilidade e o Impacto: O *tfsec* inspecionou o arquivo `infra/main.tf` e bloqueou o *pipeline* sinalizando vetores de risco críticos. O *Security Group* autorizava o

tráfego de entrada e saída por blocos CIDR 0.0.0.0/0, o que deixaria a aplicação do Helpdesk abertamente exposta a varreduras (*port scans*) de toda a internet. Adicionalmente, o volume raiz de armazenamento (EBS) não possuía a diretiva de criptografia, permitindo que a extração física do disco comprometesse a base de dados em repouso.

A Correção: O código Terraform foi refatorado. As regras de *Ingress* e *Egress* do Firewall foram encapsuladas estritamente para sub-redes da Intranet (10.0.0.0/8). O bloco de instâncias da AWS recebeu os modificadores `encrypted = true` para o armazenamento e a obrigatoriedade de uso do *token* IMDSv2 (`http_tokens = "required"`) para acessar metadados, neutralizando tentativas de ataques do tipo *Server-Side Request Forgery* (SSRF).

4.2 Falha 2: Ausência de Cabeçalhos de Segurança (OWASP ZAP)

A Vulnerabilidade e o Impacto: O ataque dinâmico disparado localmente no *Job* revelou que o servidor Nginx, responsável por servir a aplicação, vazava explicitamente a versão do seu sistema operacional (**Server: nginx/1.25.x**) na resposta HTTP. Mais grave ainda foi a constatação da total ausência do cabeçalho Anti-clickjacking (**X-Frame-Options**). A falta desta diretiva permitiria que atores maliciosos incorporassem a tela do Helpdesk em *iFrames* invisíveis de terceiros para roubar as credenciais e as ações dos funcionários logados no sistema.

A Correção: Em vez de alterar pontualmente a aplicação, o *hardening* foi implementado na raiz da infraestrutura Web. Um arquivo restrito `nginx.conf` foi acoplado à construção do *Dockerfile* do Front-end. Este arquivo suprimiu o vazamento do *software* (`server_tokens off;`) e determinou diretivas irredutíveis para os navegadores clientes: `add_header X-Frame-Options "SAMEORIGIN"` e as políticas de controle de origem (**Cross-Origin-Opener-Policy**). Após um *git push* com estas correções, a esteira foi executada novamente e o relatório subsequente confirmou a supressão definitiva da vulnerabilidade.

4.3 Falha 3: Superfície de Ataque Excessiva no Contêiner Web (Docker)

A Vulnerabilidade e o Impacto: Durante a revisão contínua das configurações de Infraestrutura como Código (IaC), identificou-se que a abordagem primária do *Dockerfile* do Front-end carregaria o código-fonte integral da aplicação, o gerenciador de pacotes (*npm*) e o *runtime* completo do *Node.js* para dentro da imagem de produção. O impacto de expor ferramentas de *build* em produção é a ampliação drástica da superfície de ataque. Caso o contêiner web sofresse um *bypass*, o invasor teria à disposição ferramentas para baixar pacotes maliciosos, executar *scripts* e visualizar o código-fonte original não-ofuscado.

A Correção: A arquitetura do *Dockerfile* do Front-end foi reescrita aplicando o padrão de *Multi-stage Builds* (Múltiplos Estágios). O primeiro estágio (**FROM node:22-slim AS build**) foi isolado unicamente para baixar as dependências e compilar a aplicação React. O segundo estágio (**FROM nginx:alpine**) utiliza uma imagem base minimalista e segura baseada em *Alpine Linux*, copiando estritamente os artefatos estáticos já compilados (`/dist`) do estágio anterior. Esta mitigação erradicou a presença do *Node.js* e do código original do ambiente produtivo, blindando o contêiner final e reduzindo as vulnerabilidades a nível de sistema operacional.

5 Decisões Arquiteturais e Desafios de CI/CD

A implementação deste projeto acadêmico exigiu escolhas para balancear conveniência e conformidade. Para mitigar o travamento sucessivo do fluxo de desenvolvimento (*Developer Experience*), a análise SCA (*npm audit*) foi configurada no modo `|| true`, agindo apenas como ferramenta de observabilidade e não como barreira impeditiva de execução. Em contrapartida, um modelo corporativo maduro exigiria o fracasso forçado da esteira (Exit Code >0) na presença de CVEs severos nas dependências.

Destaca-se também a observação do encerramento inesperado reportado pelo OWASP ZAP ao final da esteira. Os *logs* detalharam o erro `400 Bad Request / System:PublicAccess` atrelado à rotina interna de carregamento de artefatos temporários (*Create Artifact Container failed*). Uma análise profunda revelou que este não era um erro da aplicação ou do motor de segurança, mas sim uma incompatibilidade de versões obsoletas da API de armazenamento do *GitHub Actions*. O impacto real da falha foi desconsiderado com a implementação pontual do comando `continue-on-error: true`, uma vez que a etapa crucial (geração do relatório e inserção automatizada via *Issues*) ocorria nos segundos anteriores à falha técnica de carregamento, atestando a robustez funcional do pipeline.

6 Conclusão

Este trabalho atesta que, na engenharia de sistemas contemporânea, a velocidade não deve superar a confiabilidade. Ao integrar cinco camadas automatizadas de proteção diretamente no *pipeline* de *deploy*, a prova de conceito do Helpdesk superou as falhas geradas pelo desenvolvimento acelerado, validando o modelo onde a segurança age de forma silenciosa, antecipada e contínua.